BUCHAREST UNIVERSITY OF ECONOMIC STUDIES · CYBERNETICS, STATISTICS & ECONOMIC INFORMATICS

Architecture and Evaluation of a SIEM Integrating Unsupervised Anomaly Detection and Knowledge-Grounded Language Models

An AI Threat Engine on Wazuh — anomaly ensemble · hybrid RAG · agentic CVE ingestion · fully local, fully auditable

F1 89.2% RECALL 95.9% 1,333 VECTORS CITATIONS 98.3% 100% LOCAL

Graduate: Secărea Diana Maria
Coordinator: prof. univ. dr. Cătălin Boja · Bucharest 2026

Roadmap — the loop we'll walk through

detection tells you WHAT · RAG explains WHY · the agent keeps knowledge CURRENT

01 **The problem** — rule-based SIEMs can't score, explain, or self-update

02 **Machine learning** — 16 features → dual-model ensemble → tiered verdict

03 **Retrieval-augmented generation** — grounded, cited explanations from a local LLM

04 **Agentic CVE ingestion** — the knowledge base maintains itself

05 **Evaluation & live demo** — reproducible metrics + the system running

Motivation — why add AI to a SIEM?

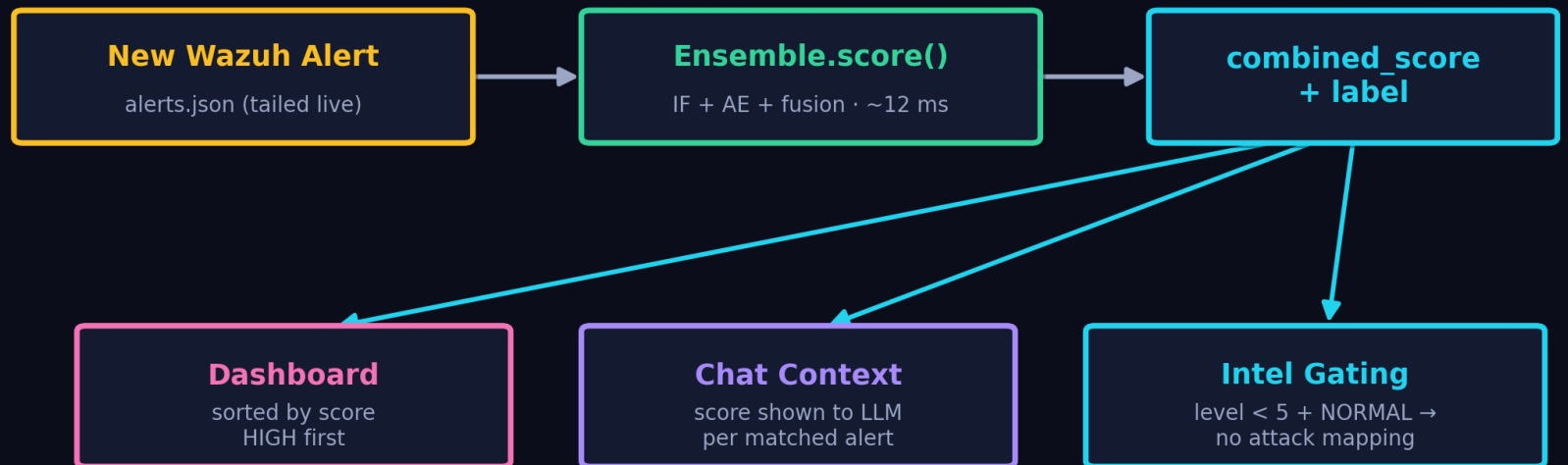
- **Alert fatigue is real** — a single host produces thousands of alerts/day; analysts triage a fraction
- **Rules only catch what they name** — signature rules miss novel behavior and low-and-slow attacks
- **Severity ≠ maliciousness** — Wazuh level 5 can be routine OR the start of a brute force — context decides
- **Knowledge is scattered** — MITRE, Sigma, CVEs, internal runbooks — nobody can hold it all in their head
- **SOC expertise is expensive** — an explainable AI layer lets one analyst do the work of a tier-1 team

Integration with Wazuh

non-invasive: Wazuh is never modified — the engine reads alongside it

- **Tap point: alerts.json** — the engine tails /var/ossec/logs/alerts/alerts.json in real time (read-only, wazuh group)
- **Every alert scored in ~12 ms** — feature extraction → ensemble → combined score + label attached to the alert
- **Daily collector + cron** — collect_training_data.py snapshots alerts for continuous retraining data
- **Flask API layer** — /api/alerts/scored, /api/chat, /api/knowledge/upload — dashboard and chat consume these
- **Wazuh metadata becomes features** — rule.level, rule.id, groups, MITRE tags feed the ML — the engine builds ON Wazuh's parsing

Score Injection — one verdict, three consumers



Technology Stack

every component is open-source and runs on one machine

MACHINE LEARNING

- scikit-learn (IF + MLP AE)
- NumPy · joblib
- StandardScaler pipeline
- custom feature extractor

RAG & LLM

- Ollama · llama3.2 (local)
- Qdrant vector DB (Docker)
- sentence-transformers (dense)
- fastembed BM25 (sparse)

DATA & BACKEND

- PostgreSQL (ledger/audit)
- Flask + SSE streaming
- Docker Compose
- Python 3.13

SECURITY BASE

- Wazuh 4.14 Manager
- MITRE ATT&CK · D3FEND
- Sigma · CISA KEV · CIS
- NVD / OTX feeds

Zero cloud dependencies → suitable for air-gapped SOCs, no per-token cost, GDPR-safe by construction

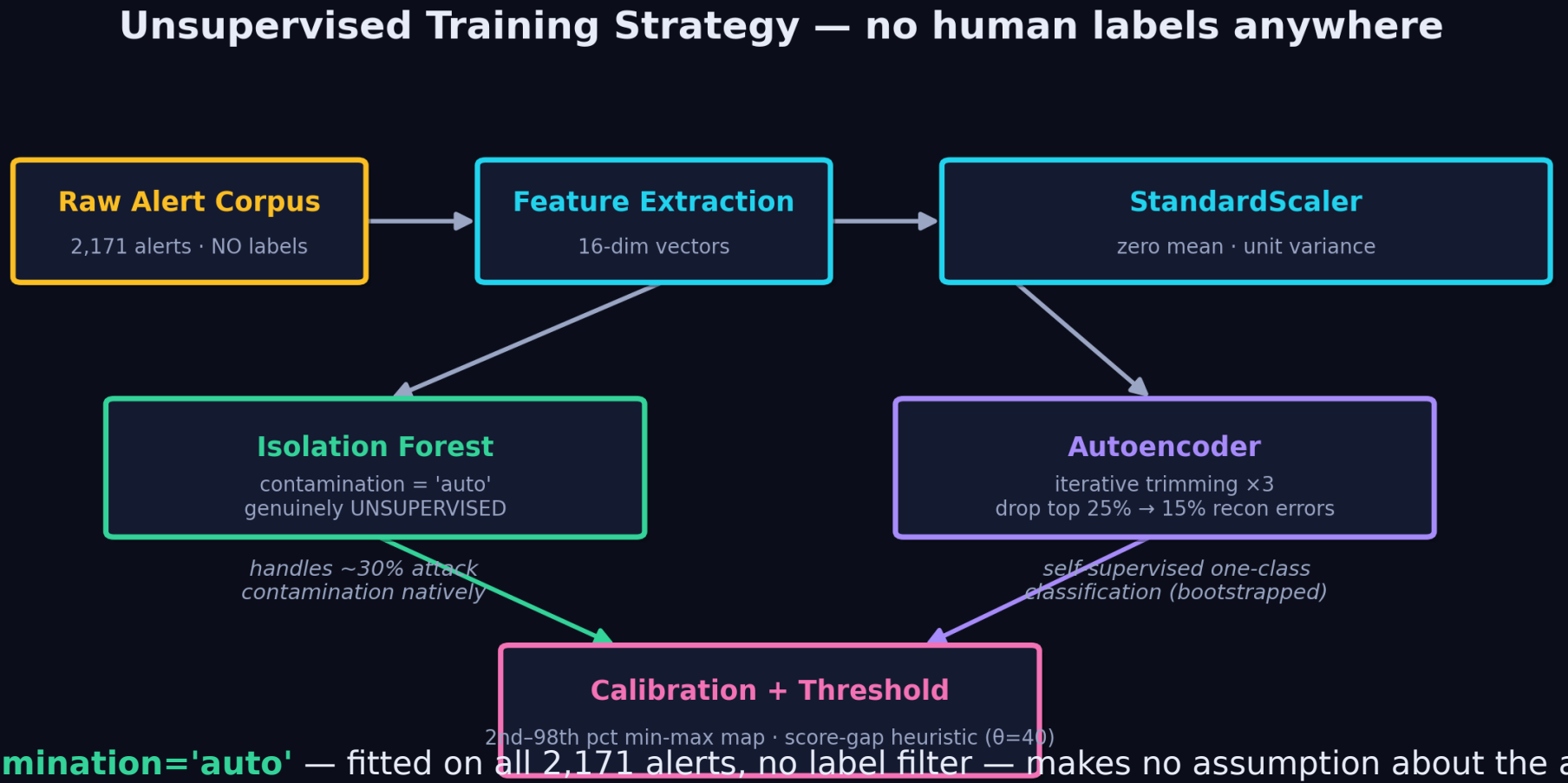
01

PART ONE · unsupervised anomaly detection

Machine Learning

Training Pipeline — unsupervised by design

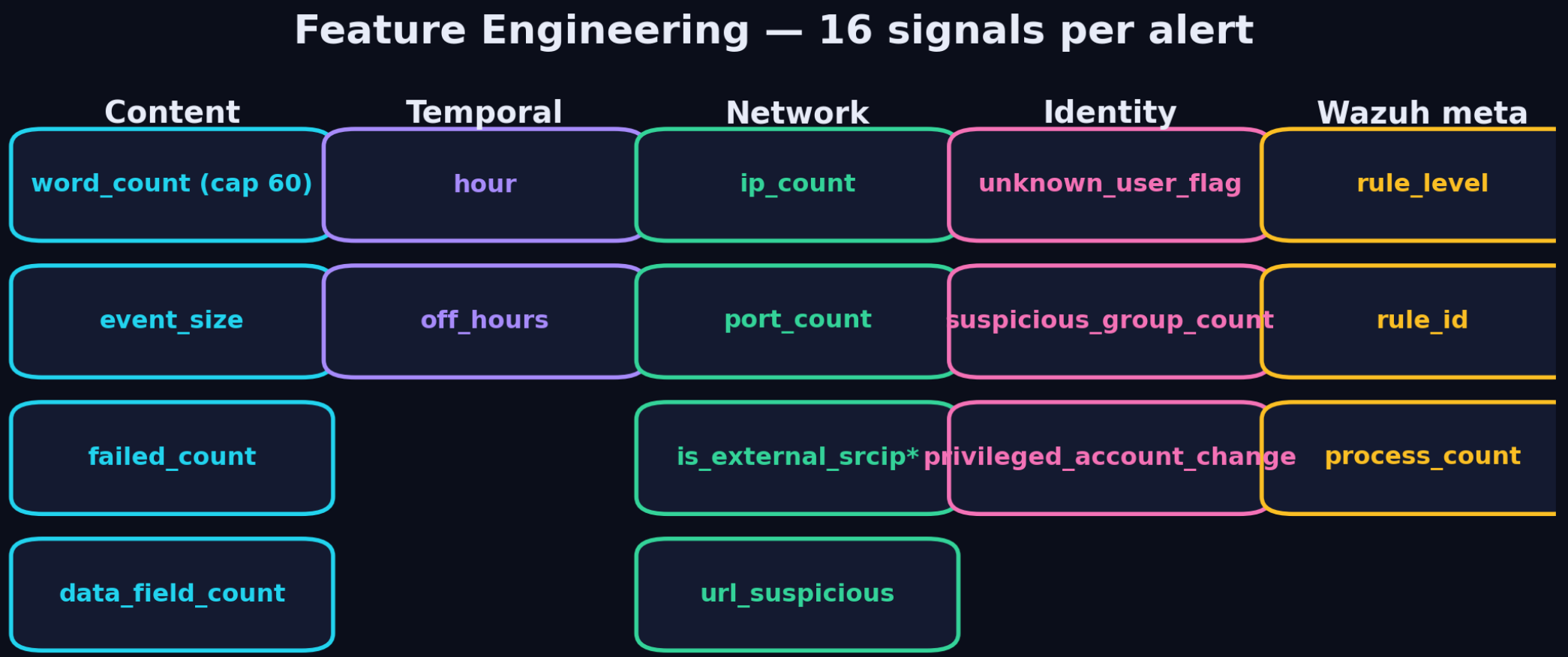
no human ever labels an alert; the models learn 'normal' from the environment itself



- **IF: contamination='auto'** — fitted on all 2,171 alerts, no label filter — makes no assumption about the anomaly proportion
- **AE: iterative trimming ×3** — ~30% attacks left in on purpose; train → drop top-25% recon errors → drop 15% → final clean fit

Feature Engineering — 16 signals per alert

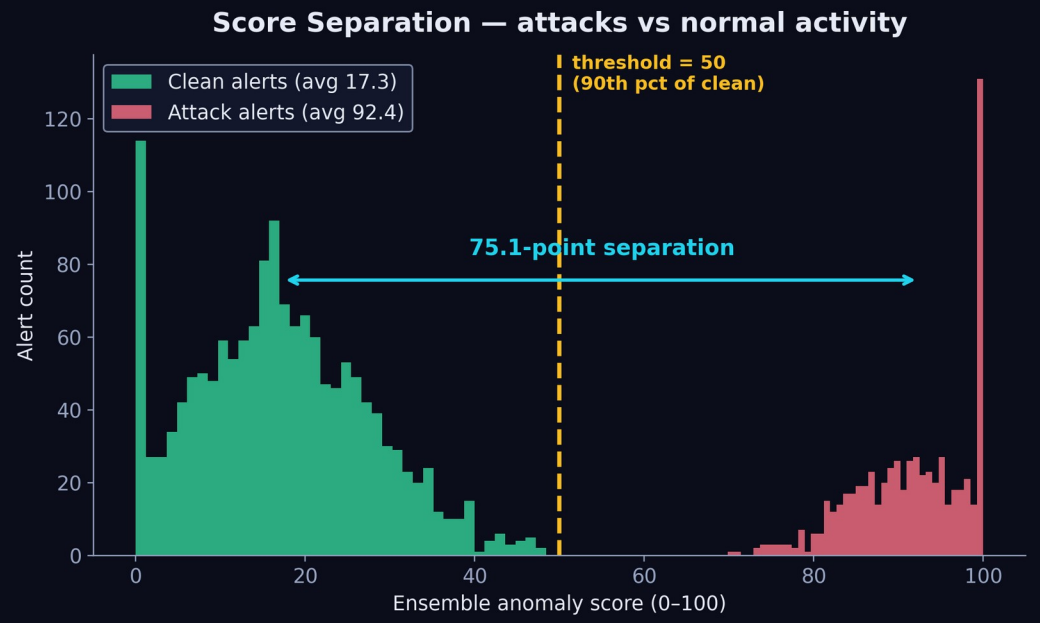
domain knowledge distilled into numbers: content, time, network, identity, Wazuh metadata



Iterated 14× against real attack simulations — e.g. MITRE-tag count was REMOVED after it caused false alarms (Wazuh tags benign SSH logins with T1078), and Cloudflare CDN ranges are excluded from the external-IP flag.

Algorithm 1 — Isolation Forest

- **Core idea** — anomalies are FEW and DIFFERENT → random splits isolate them in short paths
- **Score** — path length → anomaly score, calibrated min-max → 0-100 scale
- **Threshold** — 50 = 90th percentile of clean scores (score-based, not model.predict)
- **Why chosen** — no labels needed · handles mixed feature scales · fast (µs/alert) · robust to irrelevant features
- **Weakness** — axis-parallel splits miss subtle multivariate correlations → that's what the AE covers



75.1-point separation between attack (92.4) and clean (17.3) average scores

Algorithm 2 — Autoencoder

- **Architecture** — 16 → 8 → 4 → 8 → 16 bottleneck MLP (sklearn MLPRegressor)
- **Core idea** — trained to reconstruct NORMAL alerts; attacks reconstruct badly → high MSE = anomaly
- **The 4-dim bottleneck** — forces the net to learn the compressed manifold of normal behavior
- **Why chosen** — captures nonlinear feature CORRELATIONS the forest can't · complementary error profile
- **Training** — self-supervised — the input is its own target; iterative trimming purifies the data

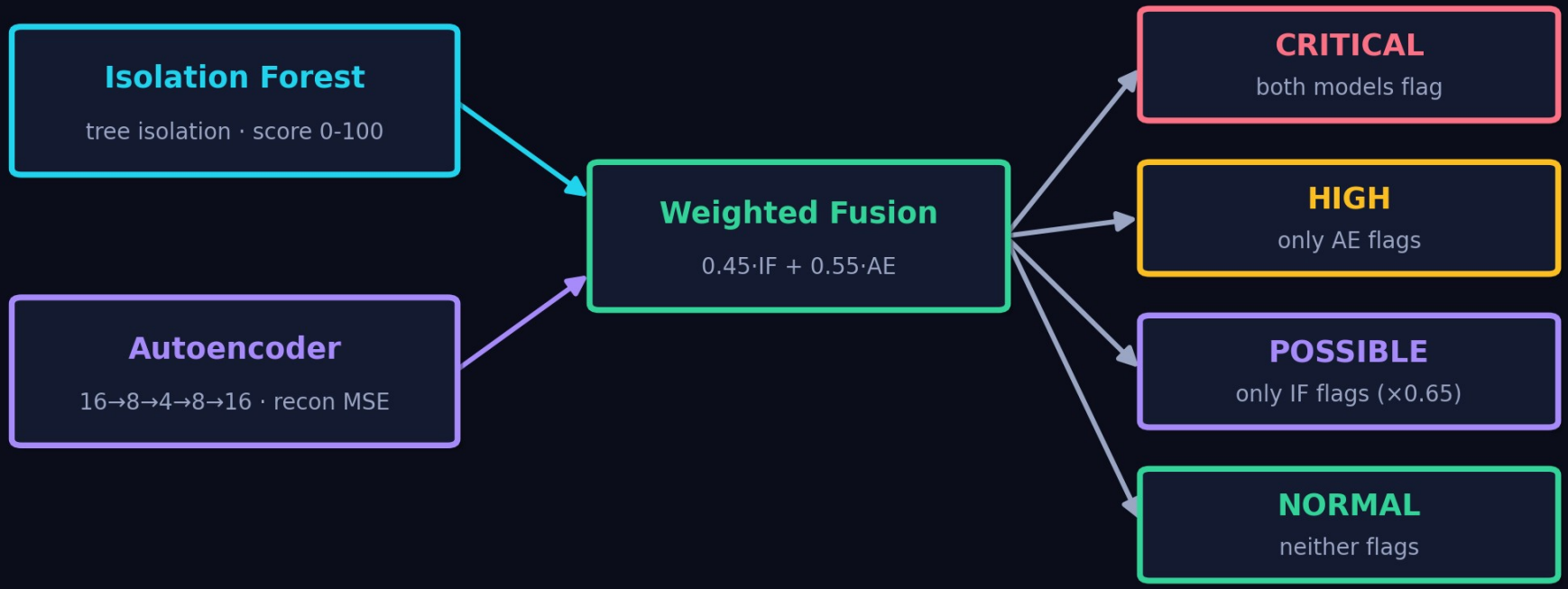
encoder → bottleneck → decoder

reconstruction error (MSE) = anomaly score

The Ensemble — fusion & label logic

the ensemble is a fixed scoring layer, not a trained model — only IF and AE are trained

Dual-Model Ensemble — score fusion & label logic



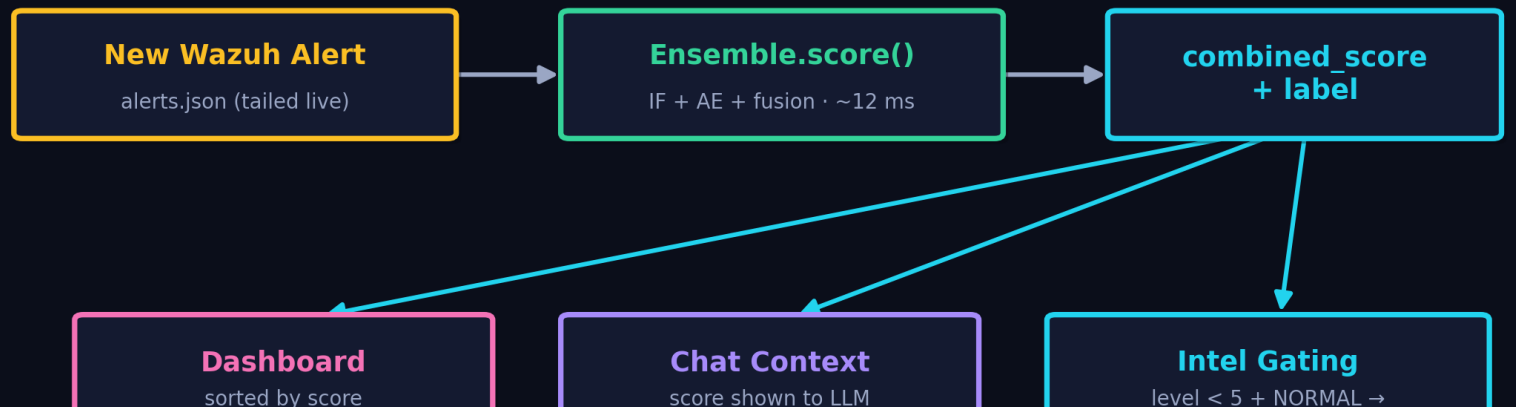
Adversarial robustness: an attacker must fool tree-based isolation AND the neural reconstruction manifold simultaneously

combined_score ∈ [0,100] · is_anomaly = label ∈ {CRITICAL, HIGH, POSSIBLE}

From Score to Action — injection into system & RAG

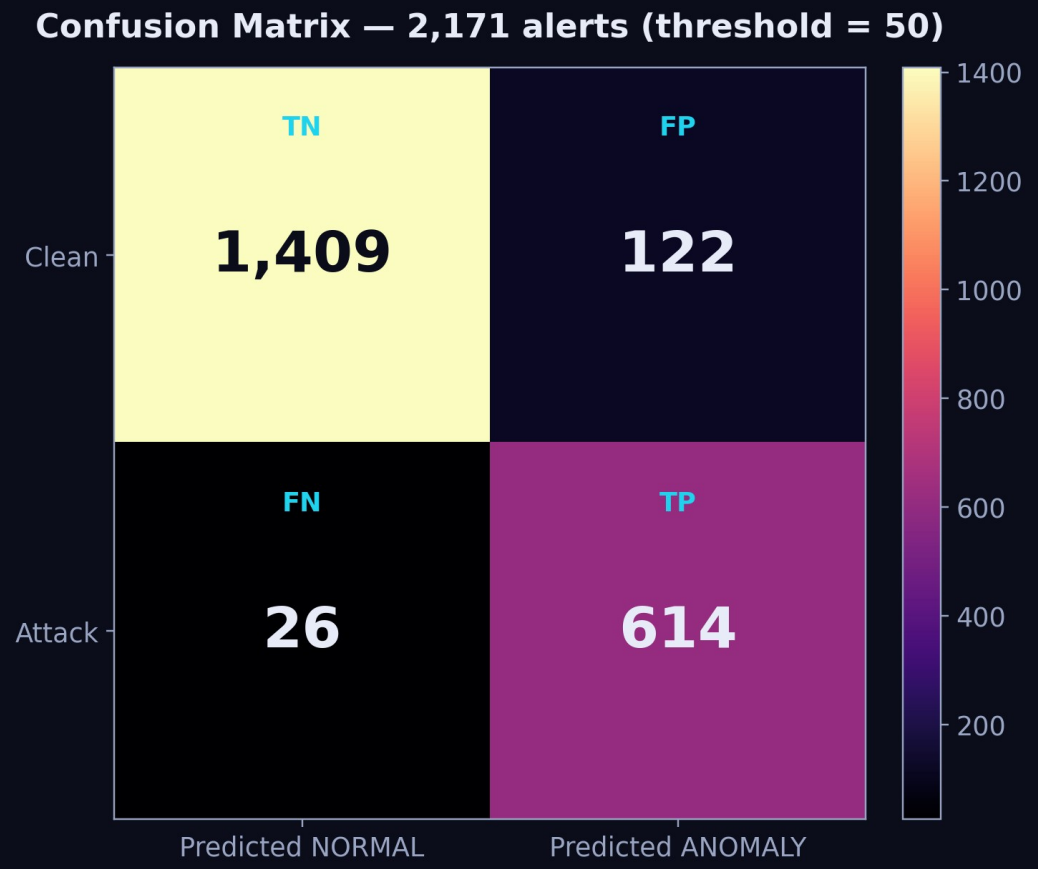
- **Dashboard** — every streamed alert carries combined_score + label; UI sorts HIGH first, benign-rule exceptions zero the score
- **Chat grounding** — matched alerts enter the LLM prompt WITH their ML verdict: 'rule 5720 · level 10 · Anomaly HIGH (94/100)'
- **Retrieval gating** — if all matched alerts are level < 5, threat-intel retrieval is SKIPPED and the LLM is told not to map to ATT&CK
- **Label → language** — CRITICAL/HIGH/POSSIBLE/NORMAL is the shared vocabulary between ML, API, UI and LLM — one verdict everywhere
- **Analyst feedback loop** — user-defined benign rules & suspicious groups adjust scoring live, no retrain needed

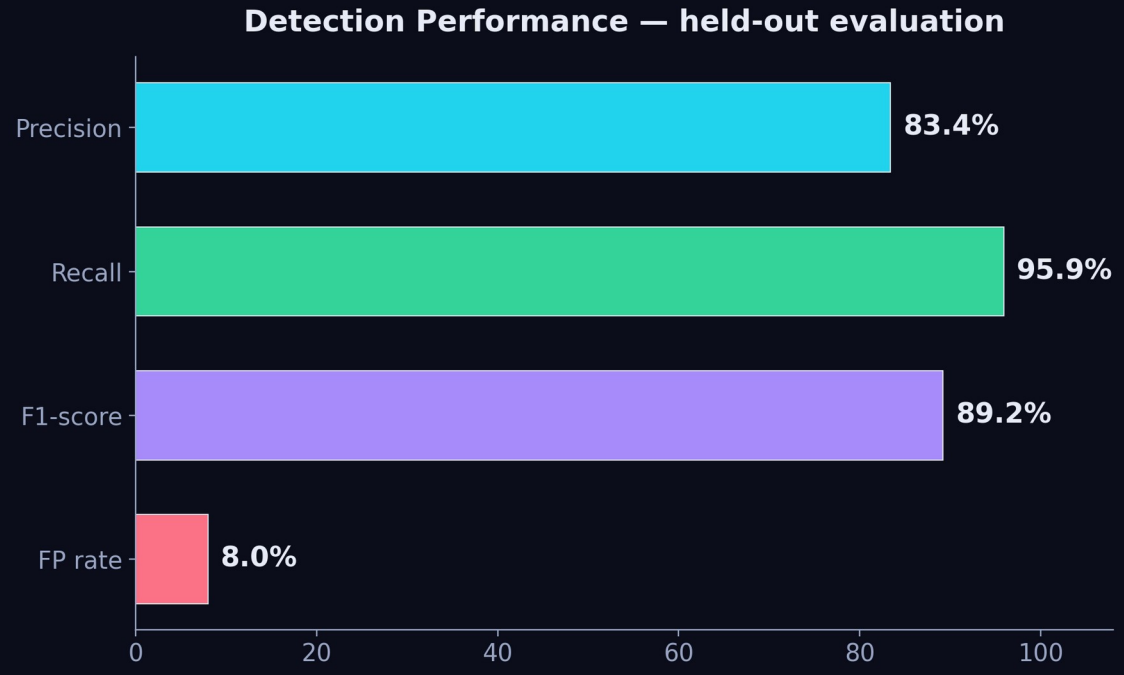
Score Injection — one verdict, three consumers



Detection Evaluation

2,171 alerts · 1,531 clean · 640 attack across 14 simulated attack families





100% detection: SSH brute force · web attacks (SQLi, XSS) · C2 & DNS exfil · reverse shells · rootkits · account manipulation.
Hardest case: single failed PAM logins (45%) — informationally identical to a typo → future work: temporal window features.

02

PART TWO · grounded, cited explanations

Retrieval-Augmented Generation

RAG — the analyst that explains

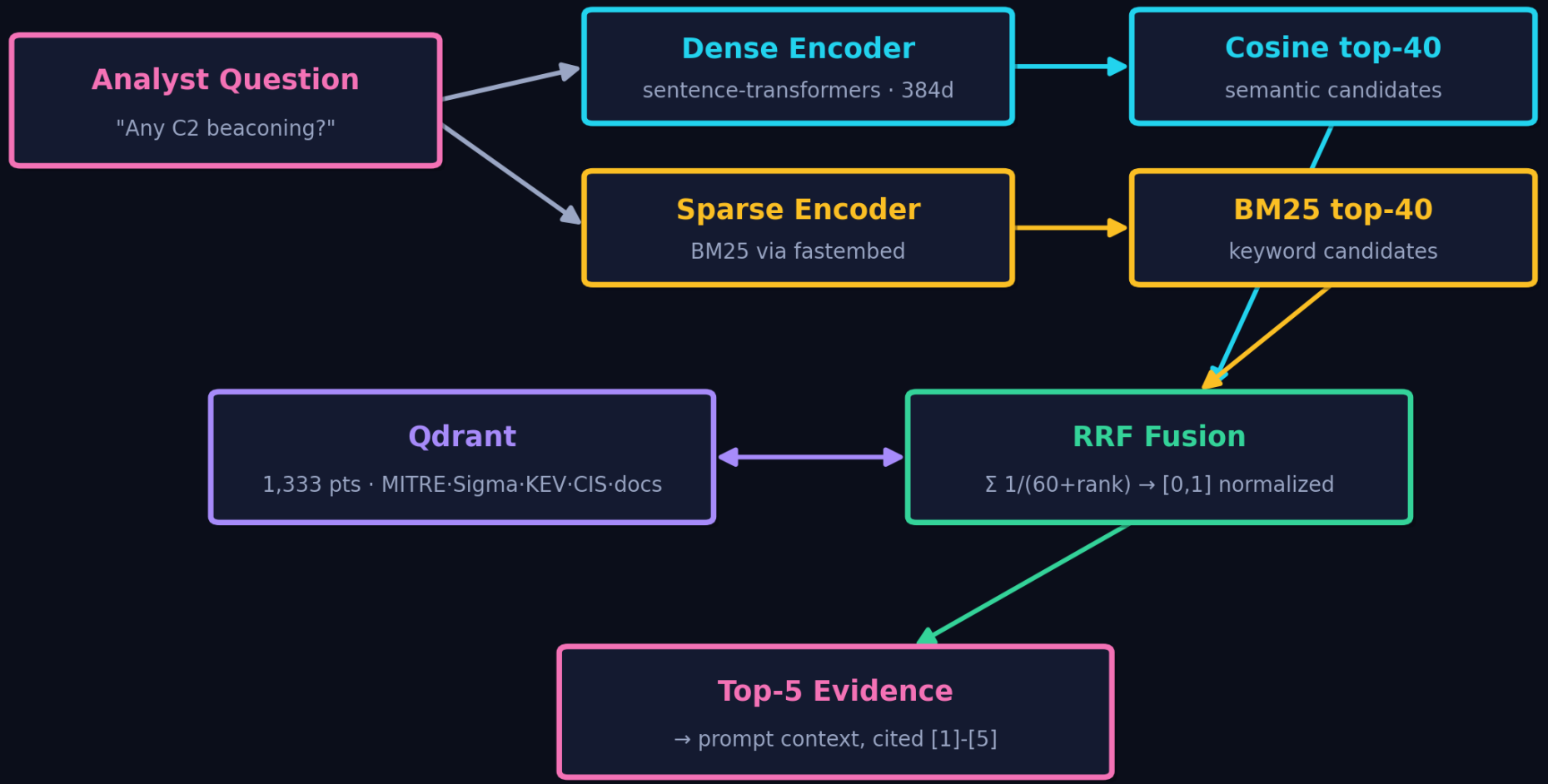
detection says WHAT is anomalous; retrieval-augmented generation explains WHY and WHAT TO DO

- **The problem with raw LLMs** — hallucination — a bare LLM will invent techniques, CVEs and IPs that don't exist
- **The RAG answer** — retrieve verified knowledge first, then force the LLM to answer FROM it, with citations [1]-[n]
- **Two grounding sources** — 1,333-point threat-intel base (MITRE·Sigma·KEV·CIS·docs) + the live Wazuh alert stream
- **ML verdicts in the loop** — anomaly labels ride along with every alert the LLM sees — detection and explanation stay consistent
- **User documents quoted verbatim** — uploaded runbooks are chunked & indexed; the analyst quotes them word-for-word with the document title
- **Session memory** — multi-turn conversations with per-session history — analysts can drill down naturally

Building the Vector Database

every knowledge item → one 'episode' → dense vector + sparse vector + payload in Qdrant

Hybrid Retrieval — dense + sparse with Reciprocal Rank Fusion



PostgreSQL — the system's memory & conscience

vectors answer 'what is similar?' — Postgres answers 'what happened, when, and why?'

threat_intel — relational mirror of every indexed knowledge item (incl. user uploads) — joinable, queryable

cve_decisions — one row per CVE the agent ever judged: base score, boost, final, decision, LLM reasoning

ingestion_log — one row per agent run: cursor used, rollup counts, status — full run traceability

pending_review view — the 4-6 quarantine queue an analyst reviews — the human-in-the-loop surface

Design principle: Qdrant is disposable (re-indexable), Postgres is the source of truth — every automated decision is auditable and reversible.

Answer Generation with grounded knowledge

PROMPT ANATOMY

SYSTEM: analyst persona + citation rules

+ anti-hallucination: low-level NORMAL alerts must NOT be mapped to attack techniques

+ quote user documents verbatim, cite [n]

=== Threat Intelligence Context ===

[1]-[5] retrieved episodes (RRF top-5)

=== Wazuh Alert Logs (matches) ===

alerts + ensemble verdicts (HIGH 94/100)

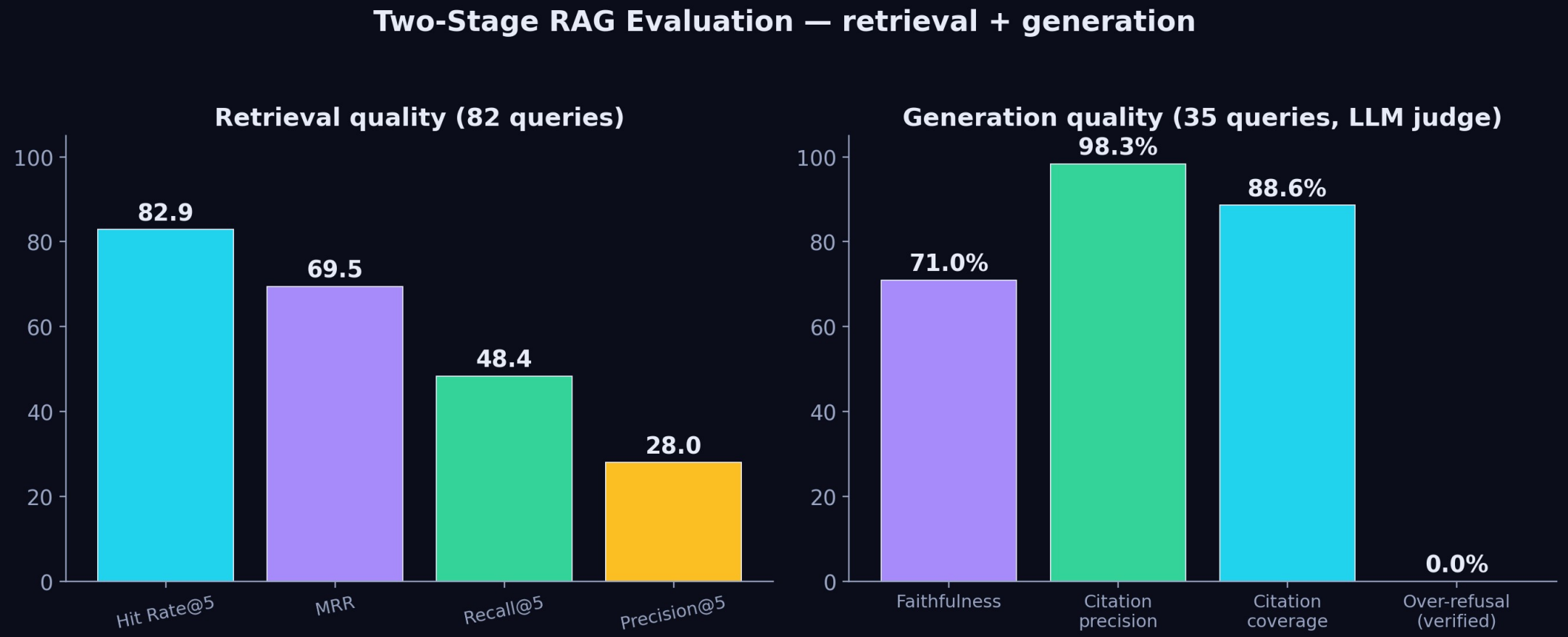
USER: the analyst's question

→ llama3.2 · streamed via SSE, token by token

- **Level-gated retrieval** — intel only fetched when an alert is level ≥ 5
- **Citations enforced** — [n] markers verified against sources in eval
- **Sources returned to UI** — every reply ships its evidence chips
- **Refusal is a feature** — no evidence → say so, never invent
- **Streaming UX** — SSE tokens — answers appear as they're generated

Evaluating the RAG — two stages

retrieval quality and generation quality are different failure modes — measure both



Honest findings: cross-encoder reranking HURT domain retrieval (removed) · weak categories identified (APT, C2 recall) · over-refusal regex flagged 8/35 but manual verification showed 0 true refusals — metrics need human validation.

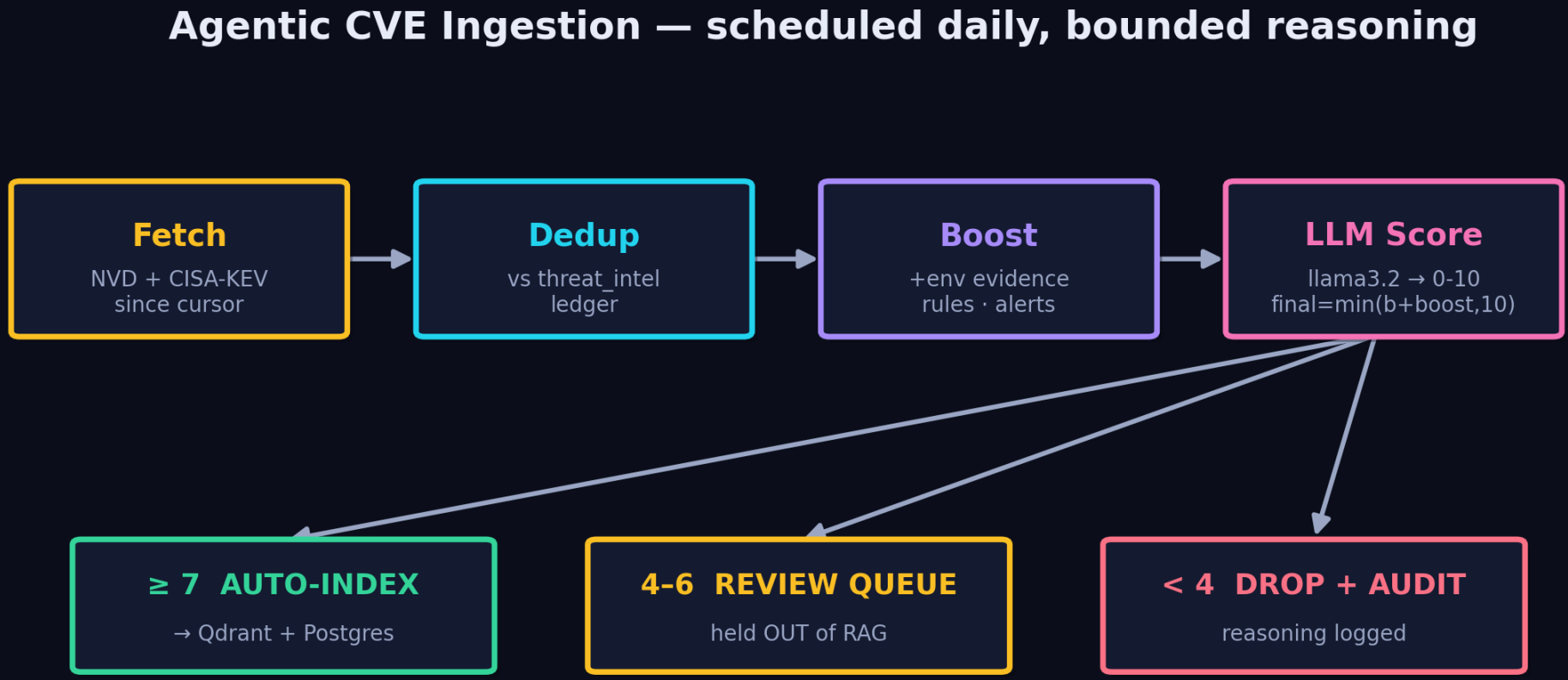
03

PART THREE · the knowledge base maintains itself

Agentic CVE Ingestion

Agentic Workflow — autonomous CVE ingestion

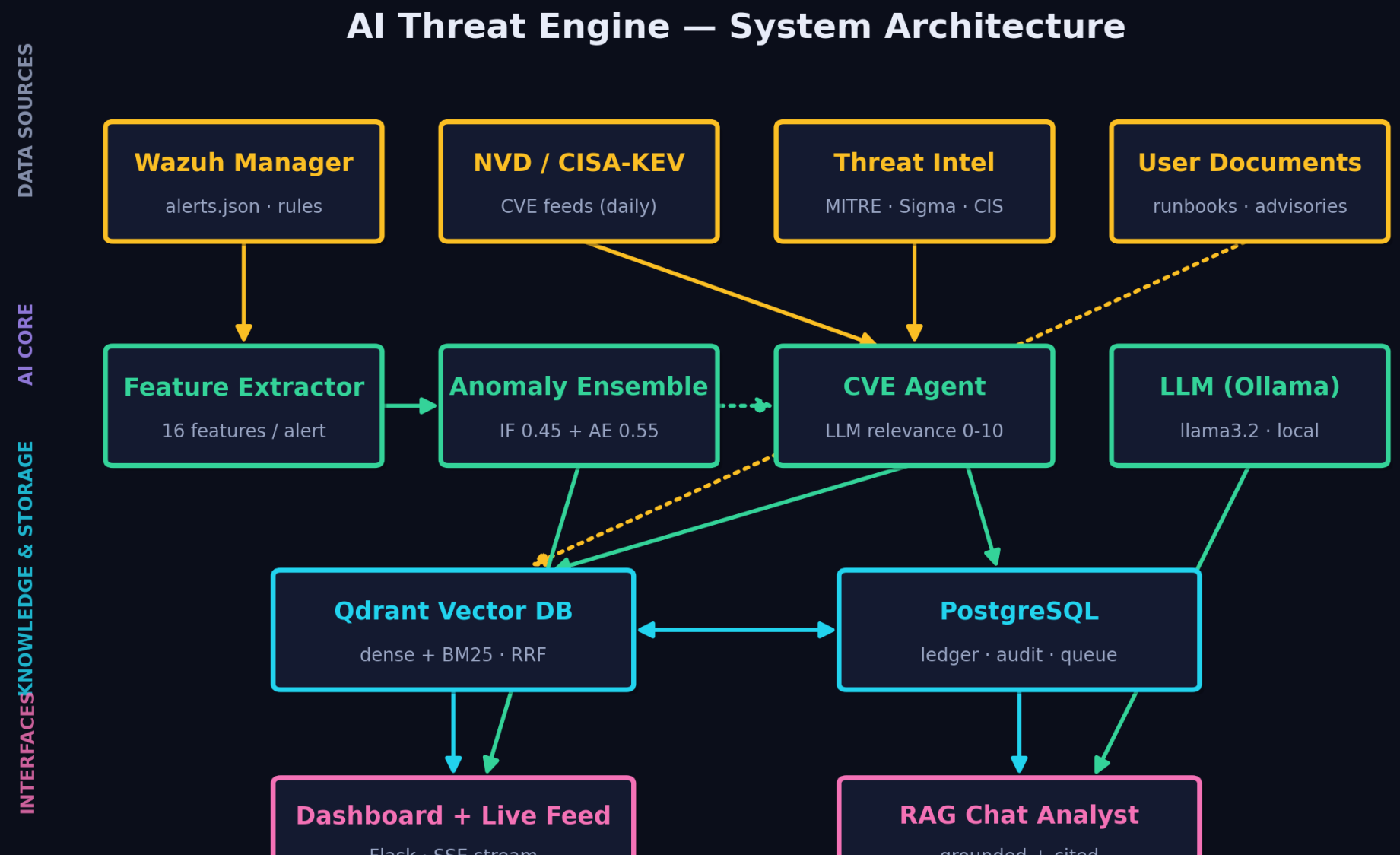
the knowledge base updates itself daily — with bounded reasoning and full audit



Every decision lands in the cve_decisions ledger — reversible, auditable, no silent RAG pollution

Design: fixed control flow, ONE bounded LLM step (relevance 0-10) · environment boost from local rules & alert history · LLM failure → conservative score 5 → review queue, never silent drop · duplicates skipped against the Postgres ledger.

The Whole System — architecture



Live Demo — the system running

ordered most-important-first: real attack → detection → grounded explanation → the ML → the agent

▶ **Real attack + attack-origin map**

Populate Map + Test the Shield — 14 families through the real Wazuh pipeline, scored live

Grounded RAG explanation

ask the analyst, click a source citation → open the exact indexed chunk in Qdrant

How the score was made

the ML Engine page: 16 features → Isolation Forest + Autoencoder → fused tier

Autonomous CVE agent

run it live — fetch → dedup → boost → LLM-score → index / queue / drop

Results at a glance

95.9% ATTACK RECALL	**89.2%** ENSEMBLE F1	**8.0%** FALSE-POSITIVE RATE	**75.1** SCORE SEPARATION (PTS)
82.9% RAG HIT RATE@5	**98.3%** CITATION PRECISION	**~12 ms** SCORING LATENCY	**14/14** ATTACK FAMILIES DETECTED

- **Every number is reproducible** — evaluation scripts, datasets and charts ship with the code
- **Honest limitations** — PAM single-login detection 45% · faithfulness judged by same-family LLM · FP rate has headroom

Future Work & Conclusion

- **Temporal features** — rolling-window aggregates (failures/IP/10 min) to catch low-and-slow and single-event attacks
- **Learned ensemble weights** — grid-search fusion weights on a validation split instead of fixed 0.45/0.55
- **Stronger judge model** — independent LLM for generation eval; PR-curve driven threshold selection
- **Query expansion for weak categories** — LLM-rewritten queries to lift APT / C2 retrieval recall
- **Cloud deployment** — Oracle ARM VM · Docker Compose · API gateway with auth + rate limiting (designed)

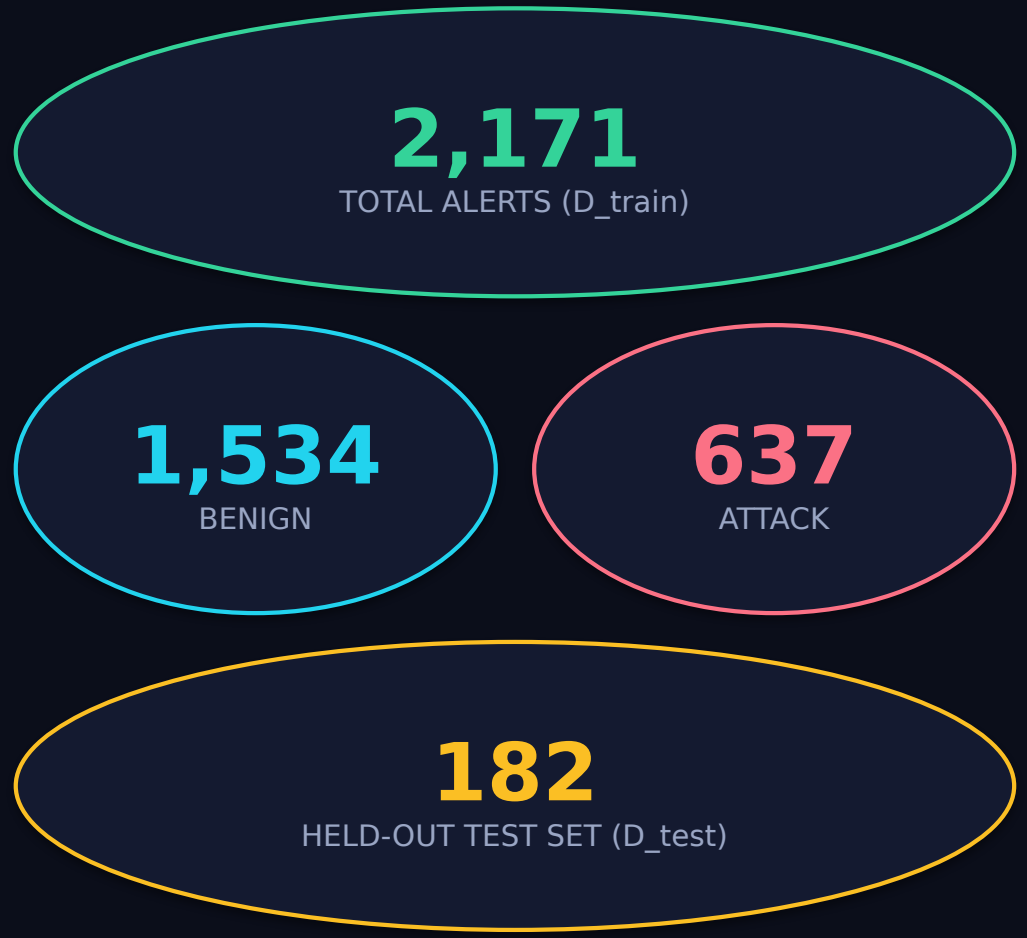
A SIEM that detects without labels, explains with citations, and updates its own knowledge — auditable, local, and reproducible end-to-end.

Thank you — questions welcome.

Training Data — two complementary sources

the model learns 'normal' locally and 'malicious' from the real world

- **Source 1 — Daily Wazuh collection** — collect_training_data.py runs via cron: reads alerts.json, filters by date, dedups by fingerprint (ts+rule+agent+msg), merges daily snapshots
- **Teaches the baseline** — authentication, rootcheck, SCA, network monitoring — what THIS deployment's normal looks like
- **Source 2 — WordPress plugin** — a live site under real attack for 2 weeks: brute force, SQLi probes, XSS payloads, automated scanning
- **Teaches malicious patterns** — genuine attack behavior the local host would rarely produce on its own



Why These Two — algorithm comparison

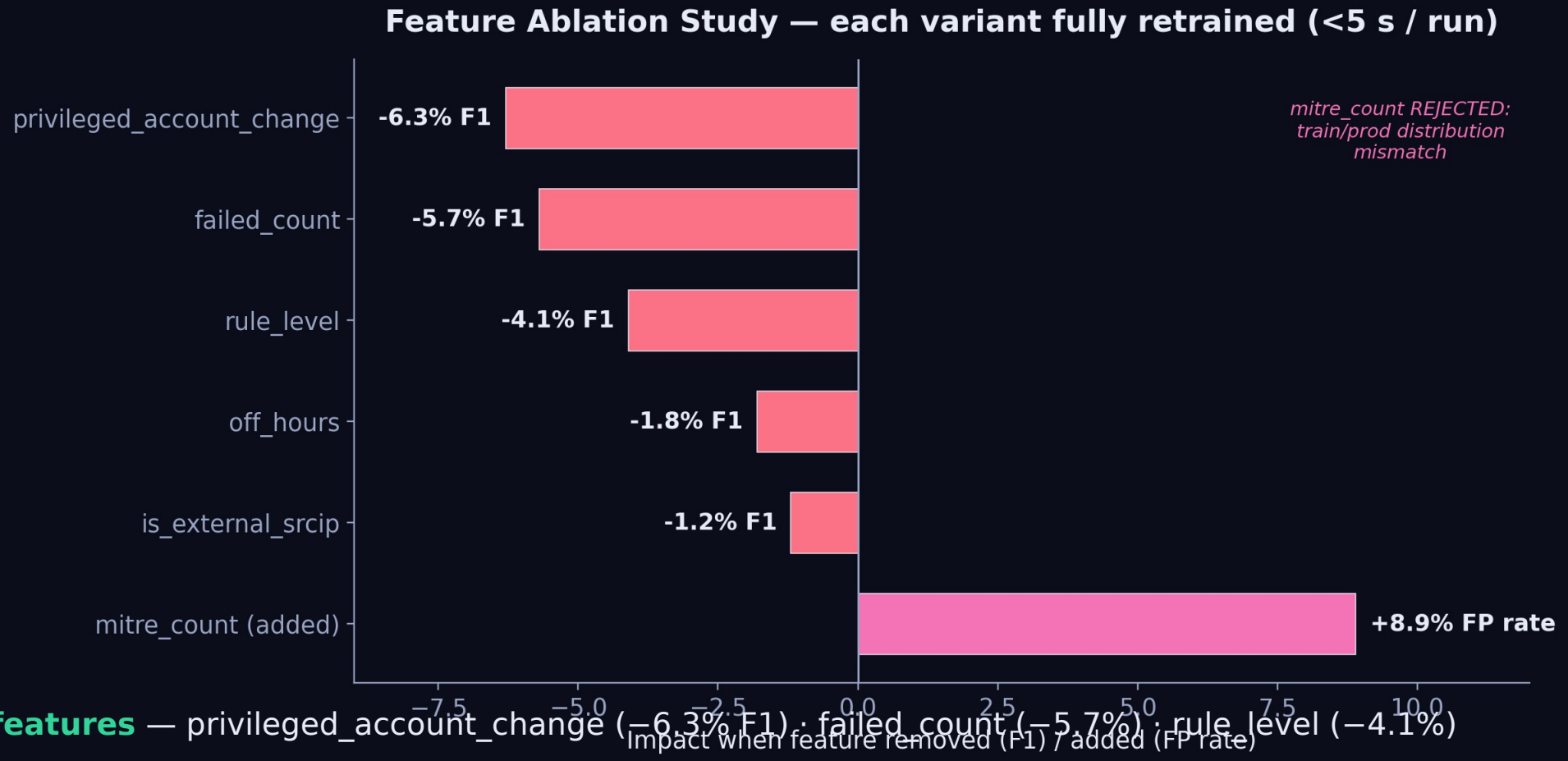
IF + AE were selected after evaluating the leading unsupervised alternatives for SIEM scoring

Method	Weakness for SIEM	Verdict
Isolation Forest	random splits can miss subtle low-dim patterns	SELECTED — best in ensemble
Autoencoder (MLP)	needs GPU / tuning; interpretability is poor	SELECTED — best in ensemble
One-Class SVM	$O(n^2)$–$O(n^3)$ training; impractical at SIEM scale	Rejected — scalability
Local Outlier Factor	$O(n^2)$ kNN; no persistable model to score new points	Rejected — no persistence
Gaussian Mixture	assumes Gaussian; poor on non-Gaussian security data	Rejected — distribution assumption
DBSCAN	eps/min_samples sensitive; produces no anomaly score	Rejected — no scoring output

Linear-time scoring, model persistence, no distribution assumptions, and native anomaly scores — only IF + AE satisfy all four.

Feature Ablation — which signals actually matter

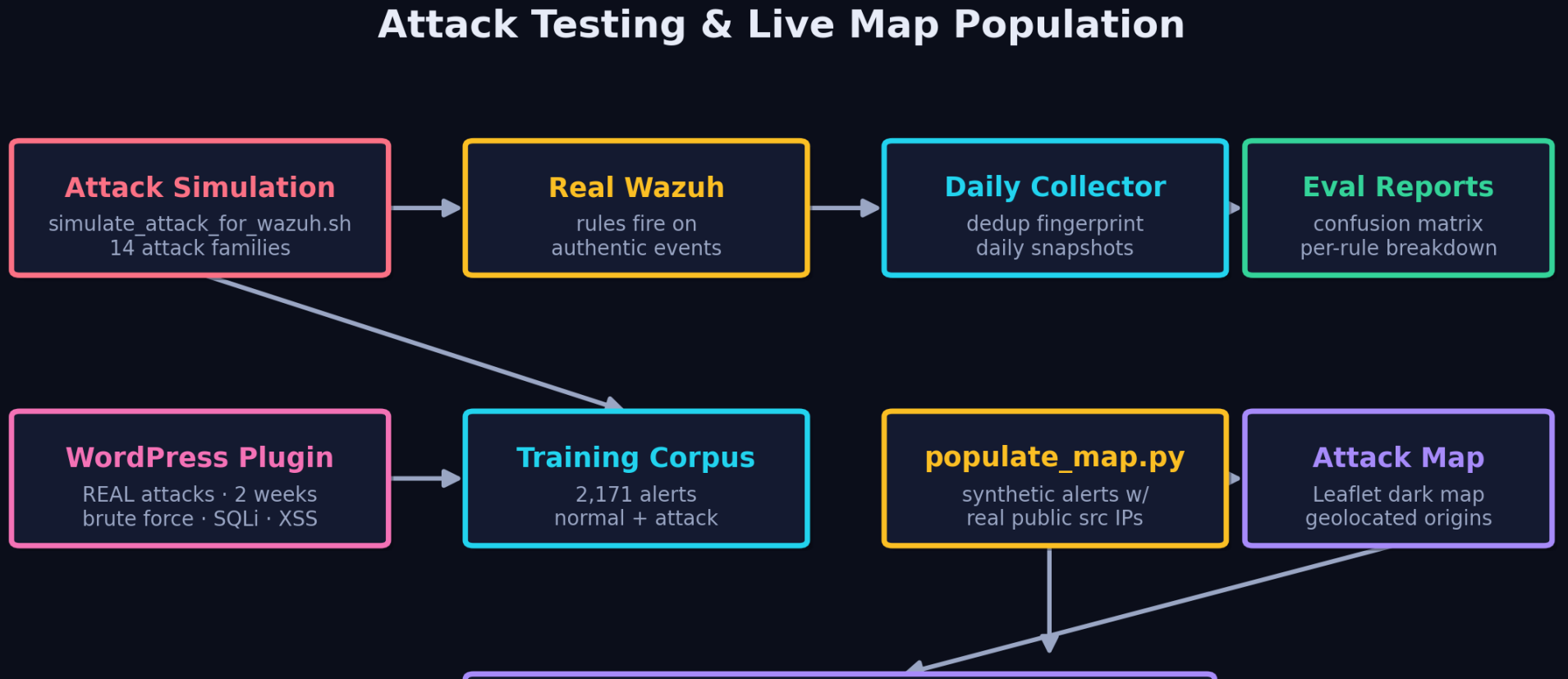
each variant fully retrained (<5 s on ~2,000 samples) and re-evaluated on the full unlabeled set



- **Top 3 features** — privileged_account_change (−6.3% F1) · failed_count (−5.7%) · rule_level (−4.1%)
- **mitre_count REJECTED** — +8.9% FP rate — train/prod distribution mismatch (Wazuh tags benign SSH logins with MITRE IDs)

Attack Testing & live map population

the pipeline is validated end-to-end against real, executed attacks — not synthetic feature vectors



- ▸ **simulate_attack_for_wazuh.sh** — runs 14 attack families as real commands → Wazuh detects → ensemble scores (trojanized files, rootkit hidden files, PAM/SU failures, port changes, MITRE-tagged events T1548/T1078/T1110/T1014)
- ▸ **populate_map.py** — injects synthetic alerts with real public source IPs → geolocated → Leaflet dark world map of attack origins

Design Choices — RAG over fine-tuning

why retrieval, why MiniLM, why RRF instead of a cross-encoder

Dimension	Fine-tune	RAG
Knowledge update	hours–days	seconds
Freshness	frozen	live
Traceability	opaque	cited
Privacy	sent to model	stays local
Cost	GPU training	embed + search

▸ **Embedding: all-MiniLM-L6-v2** — 384-dim, ~8 ms CPU — 5× faster than mpnet, keeps 90%+ quality; bigger models impractical on CPU

▸ **Chunking: 512 tokens, 25% overlap** — won a [128/256/512/1024] sweep on Recall@5 — smaller fragments concepts, larger dilutes the vector

▸ **RRF, not cross-encoder** — ms-marco reranker HURT the domain (doesn't know rule 5710 ↔ SSH); RRF trusts rank position, no learned bias

▸ **k = 5 retrieved** — maximizes Hit Rate within llama3.2's 8K window; k≥10 triggers 'lost in the middle'